Formation 7 :

Pipeline et Cross Validation

KRYPTOSPHERE®

Plan :

Rappels

Pipeline

Cross Validation

Rappels

- df = pd.read_csv(« »)

- y = df[‘Cible’]
- X = df.drop (columns = ‘Cible’)

- X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,random_state=42)

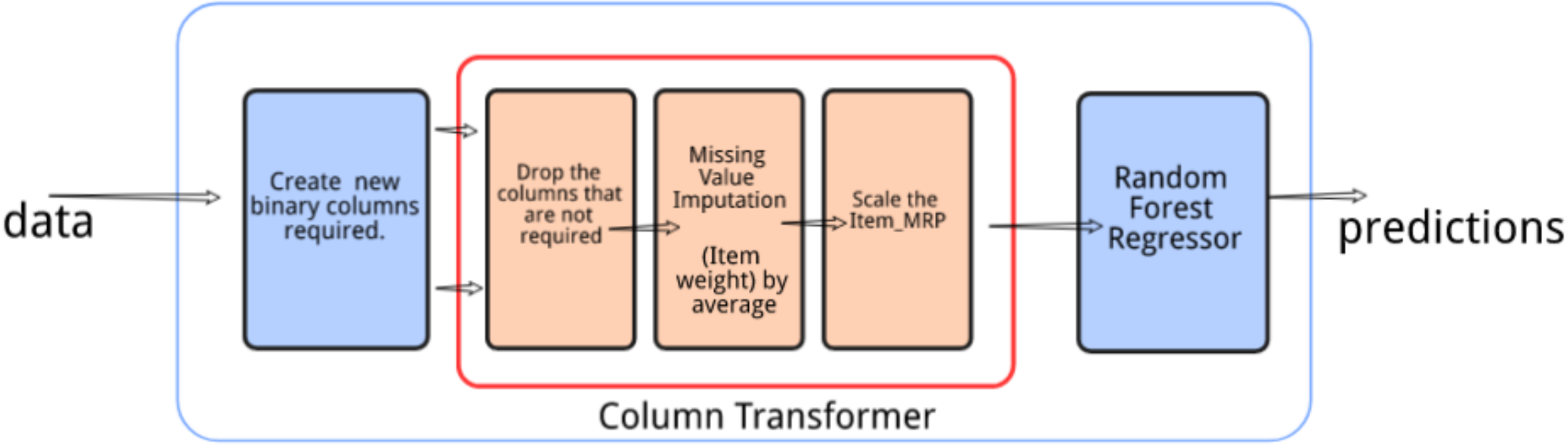
- model = xgb.XGBRegressor()
- model.fit(X_train, y_train)

- y_pred = model.predict(X_test)
- rmse = np.sqrt(mean_squared_error(y_test, y_pred))

Pipelines

pipeline.fit() when using the training data

pipeline.predict() on test data

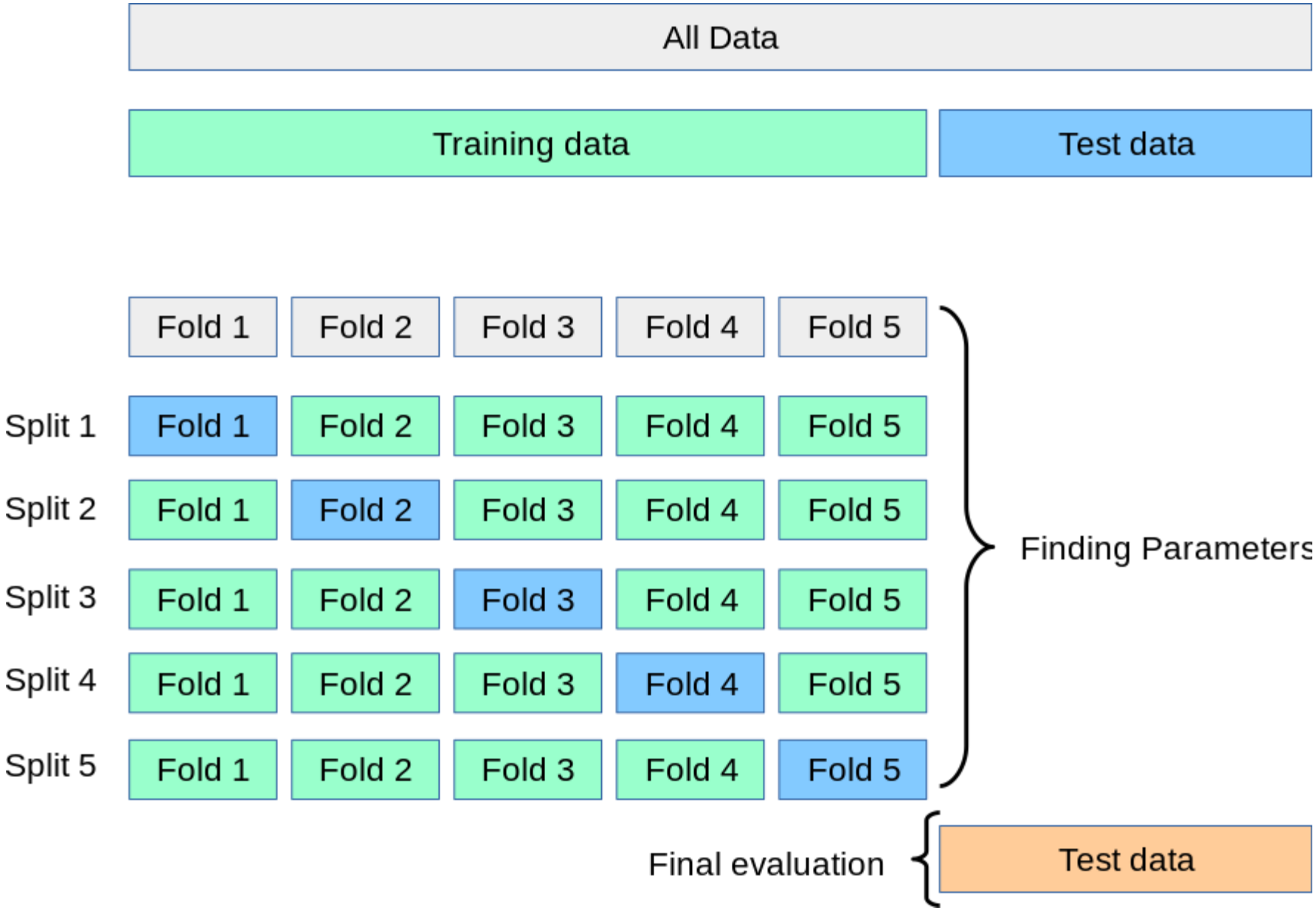


Pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import pipeline

preprocessor = ColumnTransformer(
    transformers=[
        ("nom_bloc", transformer, liste_de_colonnes),   -> transformer peut être un objet simple
                                                                                ou une pipeline
        ...
    ]
)
model_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", xgb_model),
])
```

```python
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from xgboost import XGBRegressor

# 1) Listes de colonnes (à construire à partir de X_train)
numerical_cols = [...]
categorical_cols = [...]

# 2) Préprocessing numérique
numeric_transformer = SimpleImputer(strategy="median")

# 3) Préprocessing catégoriel
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

# 4) Combiner dans un ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numerical_cols),
        ("cat", categorical_transformer, categorical_cols),
    ]
)

# 5) Définir le modèle XGBoost
xgb_model = XGBRegressor(
    n_estimators=300,
    learning_rate=0.1,
    max_depth=6,
    random_state=0,
)

# 6) Pipeline complet : Preprocessing -> XGBoost
model_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", xgb_model),
])

# 7) Entraînement
model_pipeline.fit(X_train, y_train)

# 8) Prédiction
y_pred = model_pipeline.predict(X_valid)
```

Cross Validation



- from sklearn.model_selction import cross_val_score

- score = cross_val_score(my_pipeline, X, y , cv = 5,
  scoring = mean_absolute_error)

Une utilisation de la cross validation

```python
from xgboost import XGBRegressor
from sklearn.model_selection import GridSearchCV

# Modèle de base
xgb = XGBRegressor(
    objective="reg:squarederror",
    n_estimators=500,      # upper bound, on ajustera via les hyperparamètres
    random_state=0,
)

# Grille d'hyperparamètres à tester
param_grid = {
    "max_depth": [4, 6, 8],
    "learning_rate": [0.05, 0.1],
    "subsample": [0.8, 1.0],
}

# Cross-validation k-fold utilisée comme "validation"
grid_search = GridSearchCV(
    estimator=xgb,
    param_grid=param_grid,
    cv=5,                          # 5-fold CV
    scoring="neg_mean_absolute_error",
    n_jobs=-1,                     # parallélisation
    verbose=1,
)

# On fit sur TOUT le dataset d'entraînement
grid_search.fit(X, y)

print("Meilleurs hyperparamètres :", grid_search.best_params_)
print("Meilleur score CV :", -grid_search.best_score_)

# Modèle final (déjà refitté sur tout X, y avec les meilleurs params)
best_model = grid_search.best_estimator_

# Prédictions sur un jeu de test séparé, si tu en as un
y_pred = best_model.predict(X_test)
```